ECE30030/ITP30010 Database Systems

SQL DDL

Reading: Chapter 3

Charmgil Hong

charmgil@handong.edu

Spring, 2026

Handong Global University

Announcements

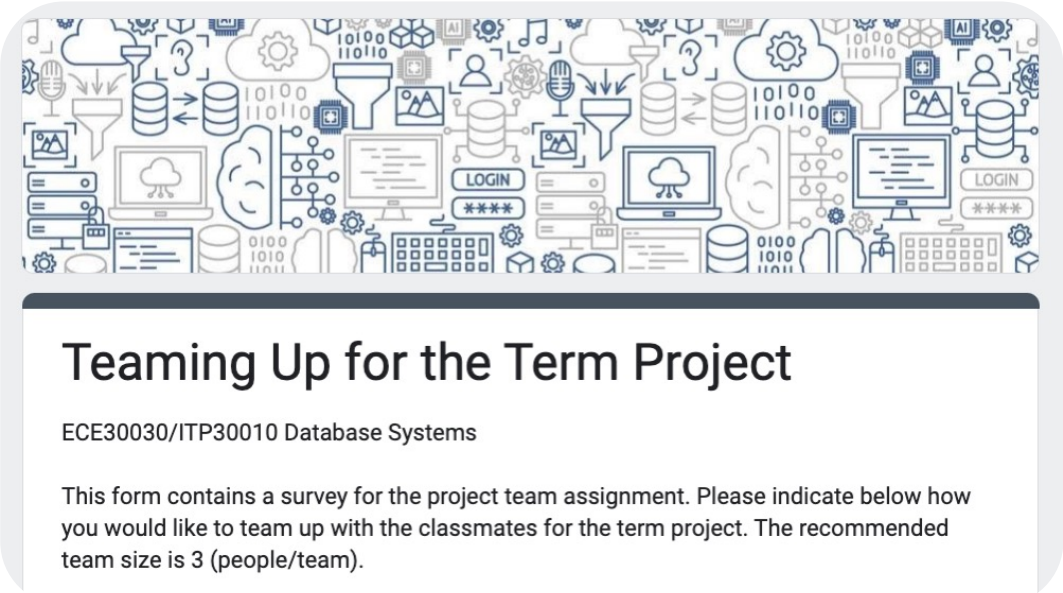
- Study material #3 is posted

- Quiz #3 is on April 23, 2026 (next Thursday)
  - Topics
    - 3 problems on SQL
    - 1 problem on ER-Model

- Midterm is on April 27, 2026 (Monday, Week #9)
  - Regular class hours
  - Closed-book
  - Topics
    - Relational data models
    - Relational algebra
    - Structured query language (DML, DDL)
    - Database design (ER-model, Normalization)

Announcements

- Make teams for the term project
  - https://forms.gle/mYJW7xU86Sv5ZD7H7 Reponse due: Thursday, April 16
  - Problem & data release: Week #8 (tentative)



Announcements

- TA office hours are available on a regular basis
  - Monday 14:00-15:00 -- Eunchan Jung (NTH217; 22100685@handong.ac.kr)
  - Tuesday 19:00-20:00 -- Hyunmo Kang (NTH217; 22100026@handong.ac.kr)
  - Thursday 14:00-15:00 -- Yunseong Choi (NTH221; yunseong.choi@handong.ac.kr)
  - Friday 14:00-15:00 -- Yujin Kim (NTH221; yujin.kim@handong.ac.kr)

  - If you find any part of the material challenging or would like additional help, please feel encouraged to visit

Aggregate Functions Examples

- Find the average salary of instructors in the Computer Science department
  - **SELECT AVG**(*salary*)
    FROM *instructor*
    WHERE *dept_name*= 'Comp. Sci.';

`AVG(salary)`
77333.333333

- Find the total number of instructors who teach a course in the Spring 2018 semester
  - **SELECT COUNT**(**DISTINCT** *ID*)
    FROM *teaches*
    WHERE *semester* = 'Spring' **AND** *year* = 2018;

`COUNT(DISTINCT ID)`
6

- Find the number of tuples in the *teaches* relation
  - **SELECT COUNT** (*)
    FROM *teaches*;

`COUNT(*)`
15

ANY: Set Comparison (= SOME)

- ANY (or SOME)
  - The **ANY** keyword compares a value with a set of values returned by a subquery
  - The condition is TRUE if the comparison holds for at least one value

  - Syntax
    - **SELECT** attribute
      FROM relation
      WHERE value <operator> **ANY** (subquery);

    - *E.g.*, Find instructors whose salary is greater than at least one instructor in the Biology department:
      - SELECT name
        FROM instructor
        WHERE salary > ANY (SELECT salary
        FROM instructor
        WHERE dept_name = 'Biology');

  - ANY ≡ SOME (same meaning)
    - **> ANY** : greater than minimum value in the set
    - **< ANY** : less than maximum value in the set

Agenda

- SQL DDL (Data Definition Language)
  - Commands
  - Data types
    - String data types
    - Numeric data types
    - **Temporal data types**
    - Large objects
  - Integrity constraints
    - Primary key constraints
    - Foreign key constraints
    - Unique constraints
    - Not null, default, check constraints

Domain Types in SQL

- SQL Data Types
  - **CHAR(*n*)**: Fixed length character string, with user-specified length *n*
    - Maximum length *n* = [0, 255]
  - **VARCHAR(*n*)**: Variable length character strings, with user-specified maximum length *n*
    - Maximum length *n* = [0, 65,535]
    - *If the length is always the same, use a CHAR-type attribute;*
      if you are storing wildly variable length strings, use a VARCHAR-type attribute
  - **TEXT**: for strings longer than the range of VARCHAR
    - TINYTEXT 0 – 255 bytes
    - TEXT 0 – 65,535 bytes
    - MEDIUMTEXT 0 – 16,777,215 bytes
    - LONGTEXT 0 – 4,294,967,295 bytes

String Data in SQL

- Difference between CHAR and VARCHAR

Value	CHAR(4)	Storage	VARCHAR(4)	Storage
‘’	‘ ‘	4 bytes	‘’	1 bytes
‘ab’	‘ab ‘	4 bytes	‘ab’	3 bytes
‘abcd’	‘abcd’	4 bytes	‘abcd’	5 bytes
‘abcdefg’	‘abcd’	4 bytes	‘abcd’	5 bytes

Domain Types in SQL

- Different R-DBMSs support different combinations of those integer types

	Bytes	MySQL	MS SQL	PostgresSQL	DB2
TINYINT	1	✓	✓		
SMALLINT	2	✓	✓	✓	✓
MEDIUMINT	3	✓			
INT/INTEGER	4	✓	✓	✓	✓
BIGINT	8	✓	✓	✓	✓

  - *C.f.*, Oracle only has a NUMBER datatype

Domain Types in SQL

- SQL Data Types
  - **NUMERIC(*p*,*d*)**: Fixed point number (exact value) with user-specified precision of *p* digits, with *d* digits to the right of decimal point
    - *E.g.*, **NUMERIC**(3,1) allows 44.5 to be stores exactly, but not 444.5 or 0.32)
    - In MySQL, **DECIMAL** is NUMERIC
  - **FLOAT**: Floating point number (approximate) with single-precision
  - **REAL, DOUBLE**: Floating point number (approximate) with double-precision

Domain Types in SQL

- DECIMAL vs INT/FLOAT/DOUBLE
  - FLOAT and DOUBLE are faster than DECIMAL
  - DECIMAL values are exact
    - Example

floats: FLOAT	decimals: DECIMAL(3,2)
1.1	1.10
1.1	1.10
1.1	1.10

- SELECT SUM(…) → DECIMAL values are precise

SUM(floats)	SUM(decimals)
3.3000000715255737	3.30

Domain Types in SQL

- SQL Data Types
  - **DATE**: ‘YYYY-MM-DD’
    - Rage: 1000-01-01 to 9999-12-31
    - *E.g.*, ‘2020-03-01’ for March 1, 2020
  - **TIME**: ‘HH:MM:SS’
    - Range: -838:59:59 to 838:59:59
    - *E.g.*, ’14:30:03.5’ for 3.5 seconds after 2:30pm
  - **DATETIME**: ‘YYYY-MM-DD HH:MM:SS’
    - Range: 1000-01-01 00:00:00 to 9999-12-31 23:59:59
  - **YEAR**: ‘YYYY’
    - Range: 1901 to 2155, or 0000 (illegal year values are converted to 0000)

Temporal Data in SQL

- SQL Data Types
  - **TIMESTAMP(*n*)**: Unix time (time since Jan 1, 1970)
    - A way to track time as a running total of seconds
    - Range: 1970-01-01 00:00:01 UTC to 2038-01-19 03:14:07 UTC
    - Typically used for logging (keeping records of all the system events)
    - *URL: https://time.is/Unix*

Temporal Data in SQL

- SQL Data Types
  - **TIMESTAMP(*n*)**: Unix time (time since Jan 1, 1970)
    - A way to track time as a running total of seconds
    - Range: 1970-01-01 00:00:01 UTC to 2038-01-19 03:14:07 UTC
    - Typically used for logging (keeping records of all the system events)
    - *URL: https://time.is/Unix*

Binary : 01111111 11111111 11111111 11110000

Decimal : 2147483632

Date : 2038-01-19 03:13:52 (UTC)

Date : 2038-01-19 03:13:52 (UTC)

On Linux

On DBMS'

Domain Types in SQL

- SQL Data Types
  - **TIMESTAMP(*n*)**: Unix time (time since Jan 1, 1970)
    - Range: 1970-01-01 00:00:01 UTC to 2038-01-19 03:14:07 UTC
    - Typically used for logging (keeping records of all the system events)

    - Depending on size *n*, the display pattern changes

	Format
TIMESTAMP(14)	YYYYMMDDHHMMSS
TIMESTAMP(12)	YYMMDDHHMMSS
TIMESTAMP(10)	YYMMDDHHMM
TIMESTAMP(8)	YYYYMMDD
TIMESTAMP(6)	YYMMDD
TIMESTAMP(4)	YYMM
TIMESTAMP(2)	YY

Domain Types in SQL

- SQL Data Types
  - **BINARY(*n*)**: binary byte data type, with user-specified length *n*
    - Contains a byte strings (rather than a character string)
    - Maximum length *n* = [0, 255]
  - **VARBINARY(*n*)**: binary byte data type, with user-specified maximum length *n*
    - Maximum length *n* = [0, 65,535]

  - **BLOB**: Binary Large OBject data type
    - TINYBLOB 0 – 255 bytes
    - BLOB 0 – 65,535 bytes (65 KB)
    - MEDIUMBLOB 0 – 16,777,215 bytes (16 MB)
    - LONGBLOB 0 – 4,294,967,295 bytes (4 GB)

Agenda

- SQL DDL (Data Definition Language)
  - Commands
  - Data types
    - String data types
    - Numeric data types
    - Temporal data types
    - Large objects
  - **Integrity constraints**
    - Primary key constraints
    - Foreign key constraints
    - Unique constraints
    - Not null, default, check constraints

CREATE TABLE Construct

- A new relation is defined using the **CREATE TABLE** command:

  CREATE TABLE *r*

  (A_1 D_1, A_2 D_2, …, A_n D_n,
  (*integrity-constraint*$_1$),
  …,
  (*integrity-constraint*$_k$))

  - *r* is the name of the relation
  - Each A_i is an attribute name in the schema of relation *r*
  - Each D_i is the data type of values in the domain of attribute A_i

- Example:

```
CREATE TABLE instructor(
        ID          CHAR(5),
        name        VARCHAR(20),
        dept_name   VARCHAR(20),
        salary      NUMERIC(8,2))
```

Integrity Constraints in CREATE TABLE

- SQL prevents any update to the database that violates an integrity constraint
  - Integrity constraints allow us to specify what data makes sense for us
- Types of integrity constraints
  - Primary key: **PRIMARY KEY** (A_1, ..., A_n)
  - Foreign key: **FOREIGN KEY** (A_m, ..., A_n) **REFERENCES** *r*
  - Unique key: **UNIQUE**
  - Not null: **NOT NULL**

- Example:

```
CREATE TABLE instructor(
        ID              CHAR(5),
        name            VARCHAR(20) NOT NULL,
        dept_name       VARCHAR(20)
        salary          NUMERIC(8, 2),
        PRIMARY KEY (ID),
        FOREIGN KEY (dept_name) REFERENCES department);
```

Declaring Keys

- **PRIMARY KEY** and **UNIQUE** constraints
  - SQL provides constraints to enforce uniqueness of attribute values
  - **PRIMARY KEY**
    - Uniquely identifies each row in a relation
    - Does not allow NULL values
    - Each relation can have only one primary key
  - **UNIQUE**
    - Ensures that attribute values are unique across tuples
    - Can allow NULL values

Declaring Keys

- Examples
  - **CREATE TABLE** instructor (
    ID **CHAR**(5) **PRIMARY KEY**,
    name **VARCHAR**(20),
    dept_name **VARCHAR**(20),
    salary **NUMERIC**(8,2)
    );
  - **CREATE TABLE** student (
    ID **CHAR**(5),
    email **VARCHAR**(50) **UNIQUE**,
    name **VARCHAR**(20)
    );
  - **CREATE TABLE** takes (
    ID **VARCHAR**(5),
    course_id **VARCHAR**(8),
    sec_id **VARCHAR**(8),
    semester **VARCHAR**(6),
    year **NUMERIC**(4,0),
    PRIMARY KEY (ID, course_id, sec_id,
    semester, year)
    );

Declaring Keys

- An attribute or list of attributes may be declared as PRIMARY KEY or UNIQUE
  - Meaning: no two tuples of the relation may agree in all the attribute(s) on the list
    - That is, the attribute(s) do(es) not allow duplicates in values
    - PRIMARY KEY/UNIQUE can be used as an identifier for each row

- Comparison: PRIMARY KEY vs UNIQUE

PRIMARY KEY	UNIQUE
Used to serve as a unique identifier for each row in a relation	Uniquely determines a row which is not primary key
Cannot accept NULL	Can accept NULL values (some DBMSs accept only one NULL value)
A relation can have only one primary key	A relation can have more than one unique attributes
Clustered index	Non-clustered index

Declaring Keys

- **FOREIGN KEY**
  - A foreign key is an attribute (or set of attributes) that references the primary key of another relation
  - Used to enforce referential integrity between tables (maintains consistency across related tables)
  - A value in the foreign key must either:
    - Match an existing value in the referenced table, or Be NULL (if allowed)
  - Examples
    - **CREATE TABLE** department (
      dept_name **VARCHAR**(20) **PRIMARY KEY**,
      building **VARCHAR**(20)
      );
    - **CREATE TABLE** instructor (
      ID CHAR(5) **PRIMARY KEY**,
      name **VARCHAR**(20),
      dept_name **VARCHAR**(20),
      FOREIGN KEY (dept_name)
      REFERENCES department(dept_name)
      );

Integrity Constraints

- **NOT NULL** – disallowing null values
  - Null values indicate that the data is not known
  - These can cause problems in querying database
  - The Primary Key columns automatically prevent null being entered
  - *C.f.*, **NULL** – can be used to explicitly allow null values

```
CREATE TABLE studio (
    ID          NUMERIC(5,0) PRIMARY KEY,
    name        VARCHAR(20) NOT NULL,
    city        VARCHAR(20) NULL,
    state       CHAR(2) NOT NULL
);
```

Integrity Constraints

- **DEFAULT** – A default value can be inserted in any column with this keyword
  - *E.g.,* **CREATE TABLE** *movies*(

```
        movie_title      VARCHAR(40) NOT NULL,
        release_date     DATE  DEFAULT sysdate  NULL,
        genre            VARCHAR(20) DEFAULT 'Comedy'
                         CHECK genre IN ('Comedy', 'Action', 'Drama')
    )
```

  - In MySQL,
    - **CREATE TABLE** *movies*(

```
        movie_title      VARCHAR(40) NOT NULL,
        release_date     DATE  DEFAULT CURRENT_TIMESTAMP  NULL,
        genre            VARCHAR(20) DEFAULT 'Comedy'
                         CHECK genre IN ('Comedy', 'Action', 'Drama')
    )
```

Integrity Constraints

- **CHECK** – Allows the inserted value to be checked
  - *E.g.*, **CREATE TABLE** *movies*(

```
        movie_title      VARCHAR(40) PRIMARY KEY,
        release_date     DATE,
        budget           INTEGER CHECK (budget > 50000)
    )
```

  - Table-level constraints can be defined; *E.g.*,
    - **CREATE TABLE** *movies*(

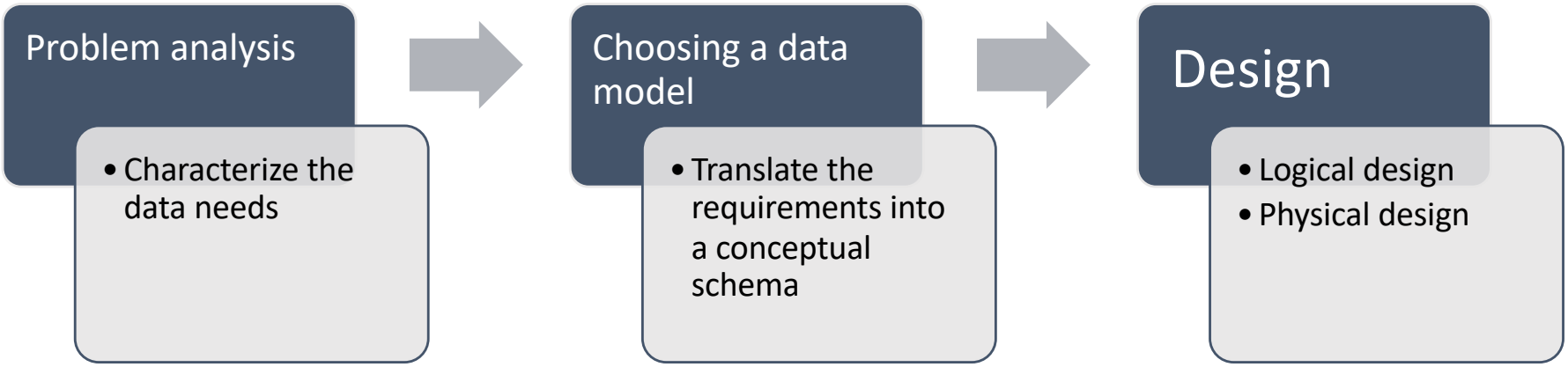
```
        movie_title      VARCHAR(40) PRIMARY KEY,
        release_date     DATE,
        budget           INTEGER CHECK (budget > 50000),
        CONSTRAINT release_date_const
        CHECK (release_date BETWEEN '01-Jan-2000' AND '31-Dec-2009')
    )
```

Agenda

- Designing a database
- E-R diagrams

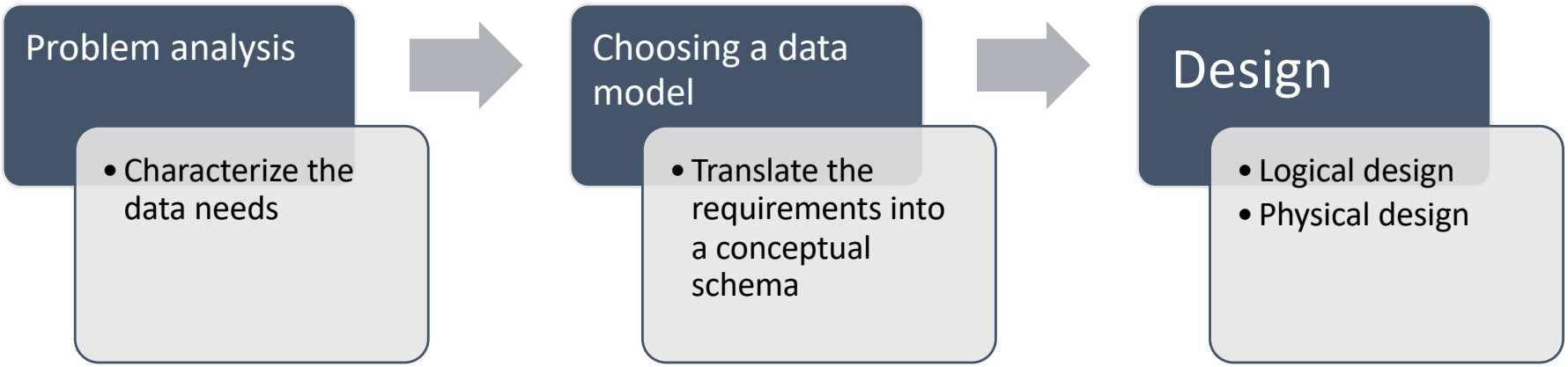
Design Phases

- Initial phase: characterize fully the data needs of the prospective database users
- Second phase: choose a data model
  - Apply the concepts of the chosen data model
  - Translate the requirements into a conceptual schema of the database
  - A fully developed conceptual schema indicates the functional requirements of the enterprise
    - Describe the kinds of operations (or transactions) that will be performed on the data



Design Phases

- Final Phase: Move from an abstract data model to the implementation of the database
  - Logical Design – Deciding on the database schema
    - Database design requires that we find a "good" collection of relation schemas
    - *Business decision – What attributes should we record in the database?*
    - *Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?*
  - Physical Design – Deciding on the physical layout of the database



Design Phases

- In designing a database schema, we must ensure that we avoid two major pitfalls:
  - Redundancy: a bad design may result in repeated information
    - Redundant representation of information may lead to data inconsistency among the various copies of information
  - Incompleteness: a bad design may make certain aspects of the enterprise difficult or impossible to model

- Avoiding bad designs is not enough. There may be a large number of good designs from which we must choose

Design Approaches

- Entity Relationship Model
  - Models an enterprise as a collection of *entities* and *relationships*
    - Entity: a "thing" or "object" in the enterprise that is distinguishable from other objects
      - Described by a set of *attributes*
    - Relationship: an association among several entities
  - Represented diagrammatically by an *entity-relationship diagram* (E-R diagram)

- Normalization Theory
  - Formalize what designs are bad, and test for them

Agenda

- Designing a database
- **E-R diagrams**
  - Mapping cardinalities
  - Primary keys in E-R models
  - Weak entity sets
  - Reduction to relation schemas

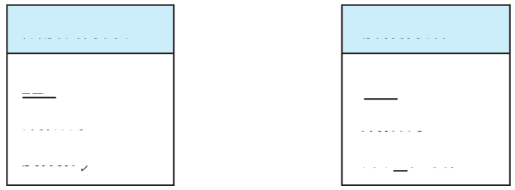
E-R Model for Database Modeling

- The E-R data model was developed to facilitate database design by allowing specification of a database schema
  - Database schema represents the overall logical structure of a database

- The E-R data model employs three basic concepts:
  - Entity sets
  - Relationship sets
  - Attributes

- The E-R model has an associated diagrammatic representation
  - E-R diagram can express the overall logical structure of a database graphically

Entity Sets



- An entity is an object that exists and is distinguishable from other objects
  - *E.g.*, specific person, company, event, plant
- An entity set is a set of entities of the same type that share the same properties
  - *E.g.*, set of all persons, companies, trees, holidays
- An entity is represented by a set of attributes; *i.e.*, descriptive properties possessed by all members of an entity set
  - *E.g.*, *instructor* = (*ID, name, salary*)
    course= (*course_id, title, credits*)
- A subset of the attributes form a primary key of the entity set; *i.e.*, uniquely identifying each member of the set

Relationship Sets

- A relationship is an association among several entities
  - *E.g.*,

    44553 (Peltier) — *advisor* — 22222 (Einstein)

    student entity — relationship set — *instructor* entity

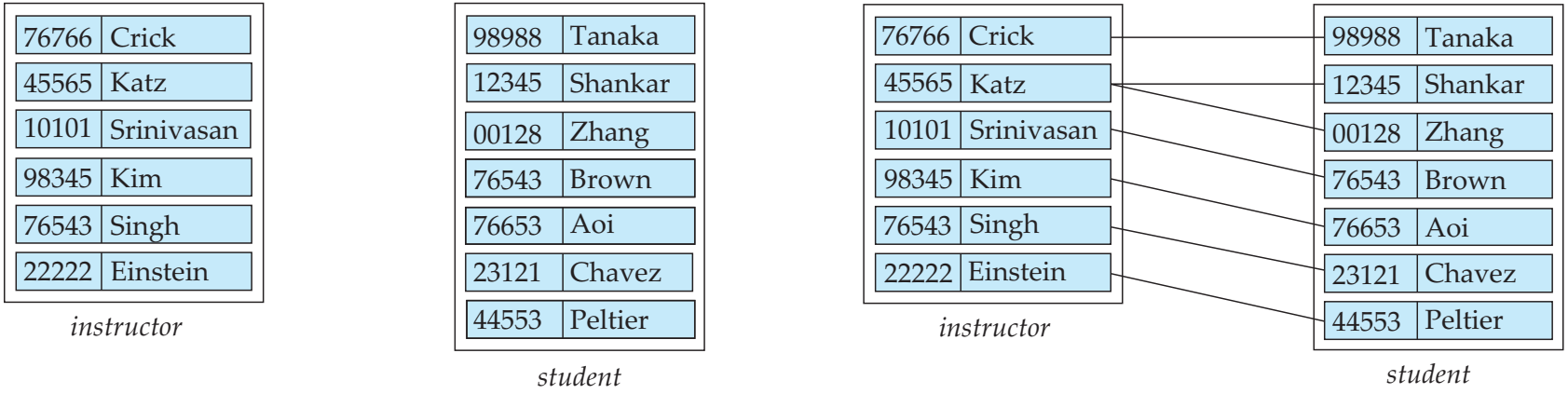
- A relationship set is a mathematical relation among $n \geq 2$ entities, each taken from entity sets

  $$\{(e_1, e_2, \ldots e_n) \mid e_1 \in E_1, e_2 \in E_2, \ldots, e_n \in E_n\}$$

  where $(e_1, e_2, \ldots, e_n)$ is a relationship
  - *E.g.*, (44553,22222) $\in$ *advisor*

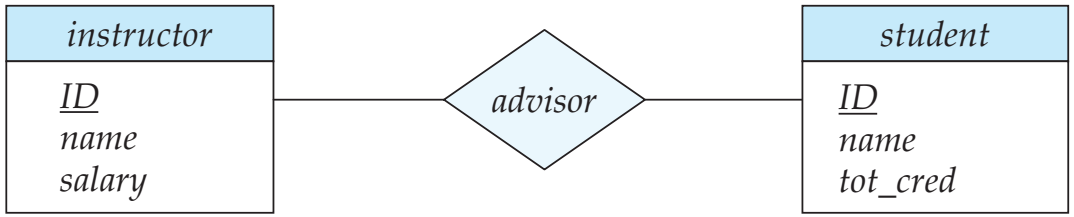
Example: Entity and Relationship Sets

- Entity Sets – *instructor* and *student*
- Relationship Sets – define the relationship set *advisor* to denote the associations between students and the instructors who act as their advisors



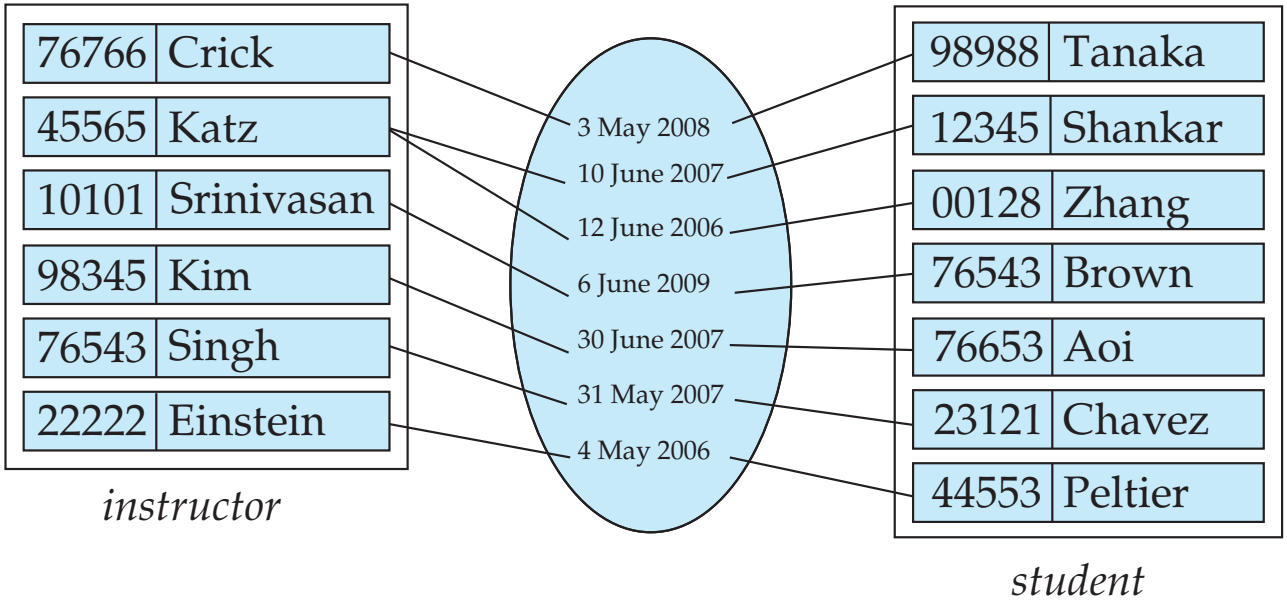
Representing Relationship Sets via E-R Diagrams

- Diamonds represent relationship sets



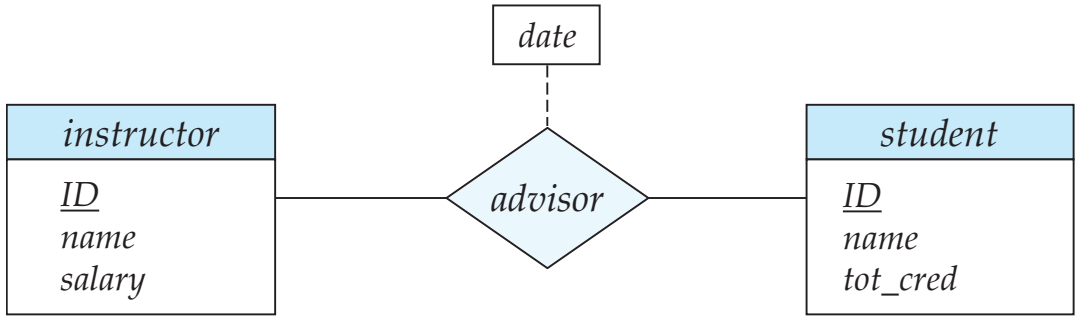
Example: Entity and Relationship Sets

- An attribute can also be associated with a relationship set
  - *E.g.*, the *advisor* relationship set between entity sets *instructor* and *student* may have the attribute *date* which tracks when the student started being associated with the advisor



Relationship Sets with Attributes

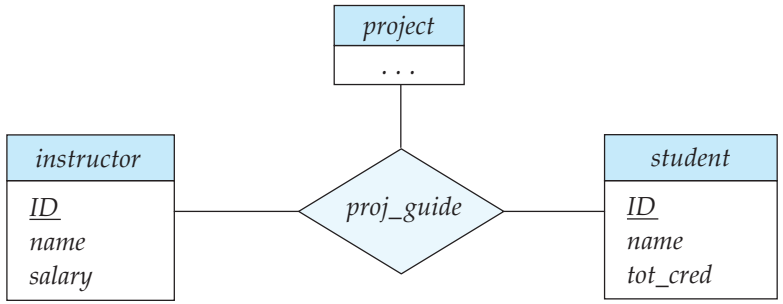
- An attribute can also be associated with a relationship set



Degree of a Relationship Set

- Binary relationship
  - Involves two entity sets (or degree two)
  - Most relationship sets in a database system are binary

- Relationships between more than two entity sets are rare but possible
  - *E.g.*, *students* work on research *projects* under the guidance of an *instructor (ternary relationship)*
  - Relationship *proj_guide* is a ternary relationship between *instructor, student,* and *project*



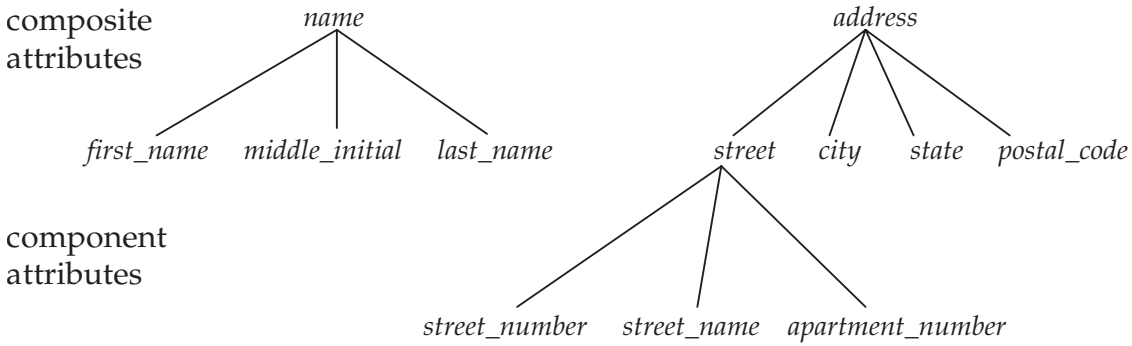
Complex Attributes

- Attribute types:
  - Simple and composite attributes
  - Single-valued and multivalued attributes
    - *E.g.*, multivalued attribute: *phone_numbers* – a person can have more than one phone numbers
  - Derived attributes: attributes that can be computed from other attributes
    - *E.g.*, age, given date_of_birth

- Domain: the set of permitted values for each attribute

Composite Attributes

- Composite attributes allow us to divided attributes into subparts (other attributes)



instructor

ID
name
  first_name
  middle_initial
  last_name
address
  street
  street_number
  street_name
  apt_number
  city
  state
  zip
{ phone_number }
date_of_birth
age ( )

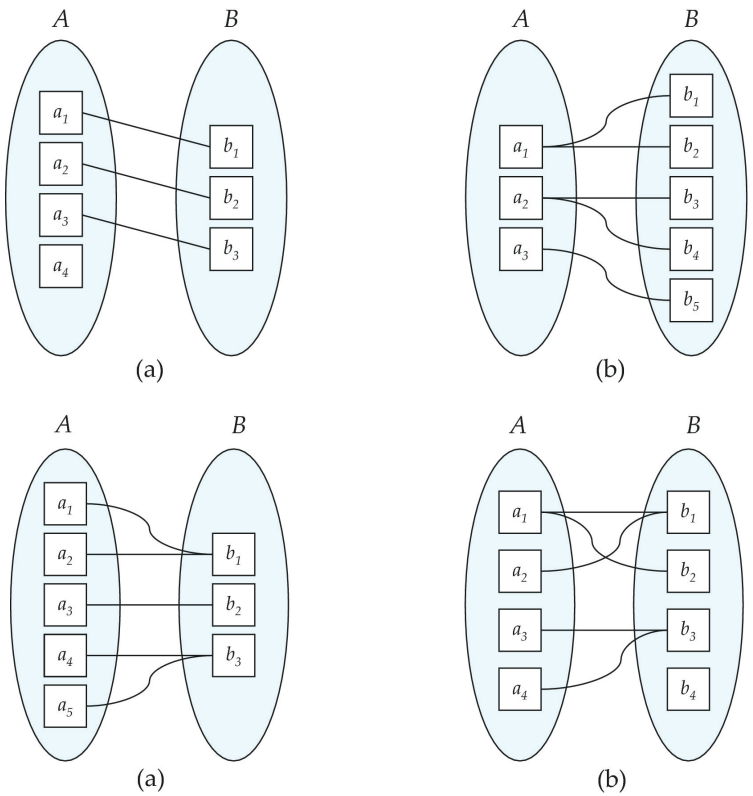
Agenda

- SQL data definition language (DDL)
- Designing a database
- E-R diagrams
  - **Mapping cardinalities**
  - Primary keys in E-R models
  - Weak entity sets
  - Reduction to relation schemas

Mapping Cardinalities

- Express the number of entities to which another entity can be associated via a relationship set
  - Most useful in describing binary relationship sets

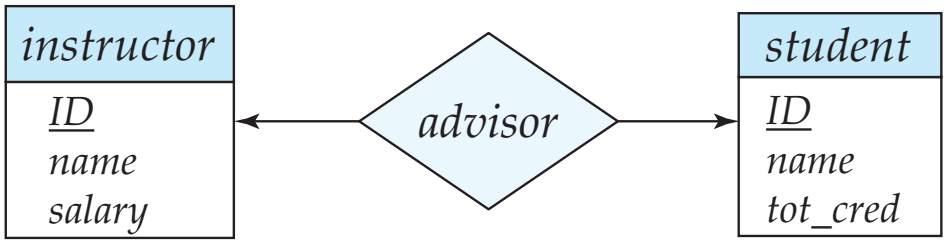
- For a binary relationship set the mapping cardinality must be one of the following types:
  - *One to one*
  - *One to many*
  - *Many to one*
  - *Many to many*



Representing Cardinalities in E-R Diagrams

- Express cardinality constraints by drawing either a directed line (→), signifying "one," or an undirected line (—), signifying "many," between the relationship set and the entity set

- One-to-one relationship between an *instructor* and a *student*:
  - A *student* is associated with at most one *instructor* via the relationship *advisor*, and *vice versa*



One-to-Many Relationship

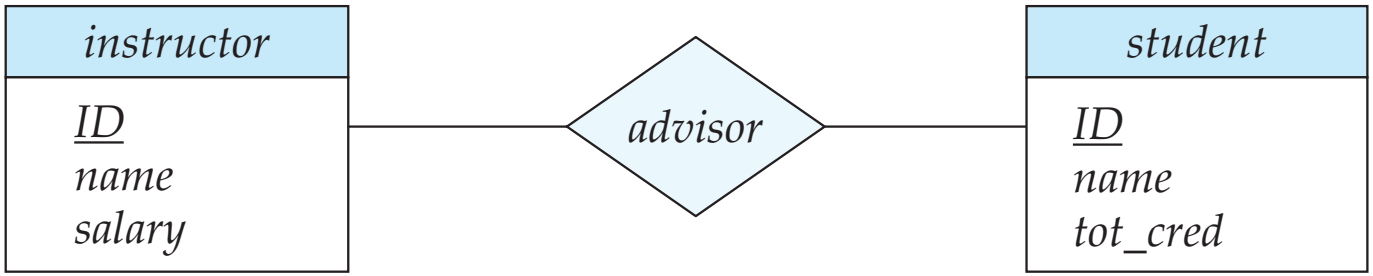
- One-to-many relationship between an *instructor* and a *student*
  - An *instructor* is associated with several (including 0) *students* via *advisor*
  - A *student* is associated with at most one *instructor* via *advisor*

Many-to-One Relationship

- Many-to-one relationship between an *instructor* and a *student*
  - An *instructor* is associated with at most one *student* via *advisor*
  - A *student* is associated with several (including 0) *instructors* via *advisor*

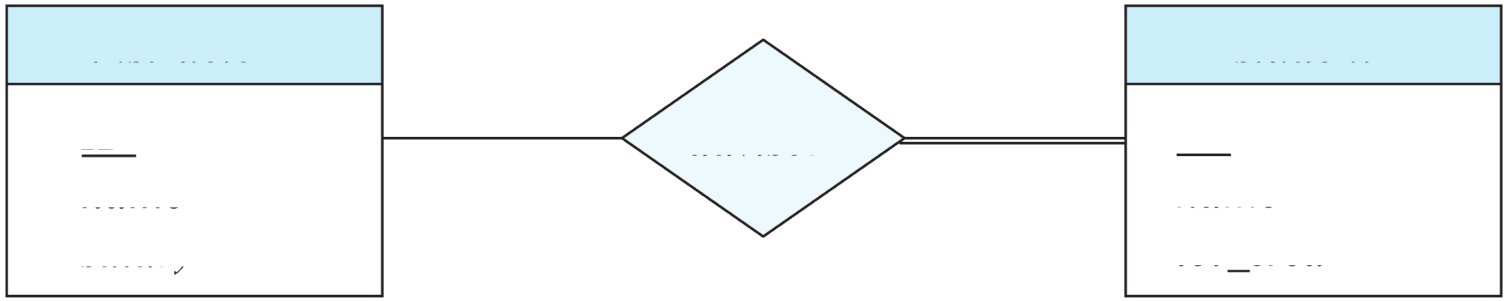
Many-to-Many Relationship

- Many-to-many relationship between an *instructor* and a *student*
  - An *instructor* is associated with several (possibly 0) *students* via *advisor*
  - A *student* is associated with several (possibly 0) *instructors* via *advisor*



Total and Partial Participation

- Total participation (*indicated by double line*): every entity in an entity set participates in at least one relationship in the relationship set



participation of *student* in *advisor r*elation is total

  - *E.g.*, Every *student* must have an associated *instructor*

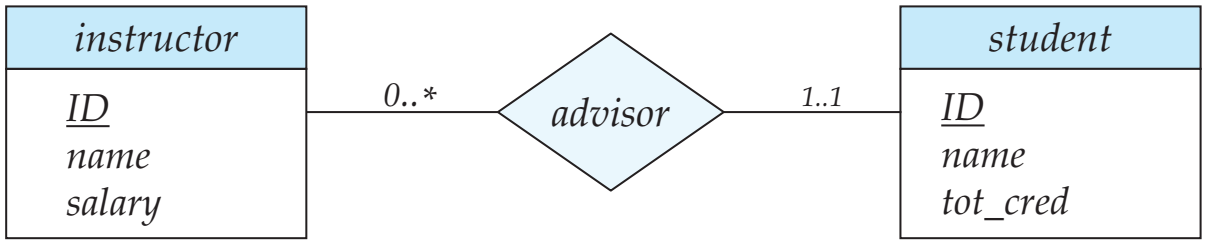
- Partial participation: some entities may not participate in any relationship in the relationship set

  - *E.g.*, Participation of *instructor* in *advisor* is partial

Notation for Expressing More Complex Constraints

- A line may have an associated minimum and maximum cardinality, shown in the form *l..h*, where *l* is the minimum and *h* the maximum cardinality
  - A minimum value of 1 indicates total participation
  - A maximum value of 1 indicates that the entity participates in at most one relationship
  - A maximum value of * indicates no limit

- Examples
  - Instructor can advise 0 or more students
  - A student must have 1 advisor; cannot have multiple advisors



Agenda

- SQL data definition language (DDL)
- Designing a database
- E-R diagrams
  - Mapping cardinalities
  - **Primary keys in E-R models**
  - Weak entity sets
  - Reduction to relation schemas

Primary Key

- Primary keys provide a way to specify how entities and relationships are distinguished
- We consider:
  - Entity sets
  - Relationship sets
  - Weak entity sets

Primary Key for Entity Sets

- By definition, individual entities are distinct
- From database perspective, the *differences among entities must be expressed in terms of their attributes*
  - The attribute values of an entity must be such that they can uniquely identify the entity
  - No two entities in an entity set are allowed to have exactly the same value for all attributes
- A key for an entity is a set of attributes that suffice to distinguish entities from each other

Primary Key for Relationship Sets

- To distinguish among the various relationships of a relationship set, use the individual primary keys of the entities in the relationship set
  - Let *R* be a relationship set involving entity sets $E_1, E_2, \ldots, E_n$
  - The primary key for *R* is consists of the union of the primary keys of entity sets $E_1, E_2, \ldots, E_n$
  - If the relationship set *R* has attributes $a_1, a_2, \ldots, a_m$ associated with it, then the primary key of *R* also includes the attributes $a_1, a_2, \ldots, a_m$

- Example: relationship set "*advisor*"
  - The primary key consists of *inrsructor.ID* and *student.ID*

Choice of Primary Key for Binary Relationship

- The choice of the primary key for a relationship set depends on the mapping cardinality of the relationship set
  - Many-to-Many relationships: The preceding union of the primary keys is a minimal super key and is chosen as the primary key
  - One-to-Many relationships: The primary key of the "Many" side is a minimal super key and is used as the primary key
    - Many-to-one relationships: The primary key of the "Many" side is a minimal super key and is used as the primary key
  - One-to-one relationships: The primary key of either one of the participating entity sets forms a minimal super key, and either one can be chosen as the primary key

Agenda

- SQL data definition language (DDL)
- Designing a database
- E-R diagrams
  - Mapping cardinalities
  - Primary keys in E-R models
  - **Weak entity sets**
  - Reduction to relation schemas

Weak Entity Sets

- A weak entity set is one whose existence is dependent on another entity, called its identifying entity
- Instead of associating a primary key with a weak entity, use the identifying entity, along with extra attributes called discriminator to uniquely identify a weak entity
  - A weak entity set does not have a primary key
  - We still need a *means of distinguishing* among an entity set
    - Discriminator of a weak entity: a set of attributes allowing such distinction
    - Primary key of a weak entity set
      = primary key of a strong entity set (which its existence depends)
      + its discriminator

Weak Entity Sets

- A weak entity set is one whose existence is dependent on another entity, called its identifying entity
- Instead of associating a primary key with a weak entity, use the identifying entity, along with extra attributes called discriminator to uniquely identify a weak entity
  - *E.g.*, Consider a *section* entity, which is uniquely identified by a *course_id*, *semester, year*, and *sec_id* → Section entities are related to course entities
    - Treat the relationship *sec_course* as a special relationship that provides extra information
    - In this case, the *course_id*, required to identify *section* entities uniquely

Weak Entity Sets

- Identifying entity
  - Every weak entity must be associated with an identifying entity;
    - That is, the weak entity set is said to be existence dependent on the identifying entity set

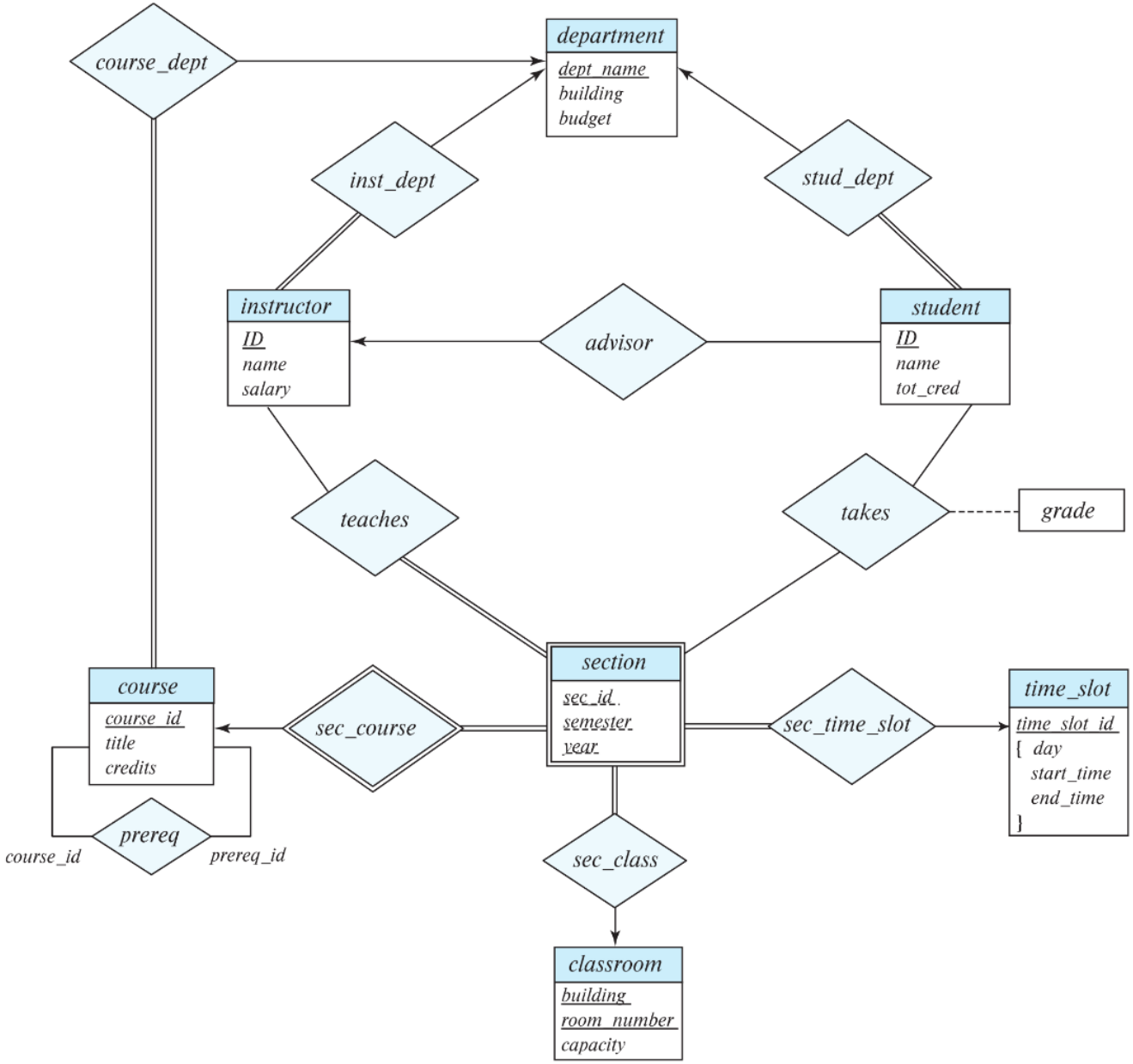
- The identifying entity set is said to own the weak entity set that it identifies
  - Identifying entity set: an entity set that has a primary key
  - Identifying entity set = strong entity set

- Identifying relationship
  - Identifying relationship: The relationship associating the weak entity set with the identifying entity set

Expressing Weak Entity Sets

- A weak entity set is depicted via a double rectangle
- Underline the discriminator of a weak entity set with a dashed line
- The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond
  - *E.g.*, Primary key for section – (*course_id, sec_id, semester, year*)

E-R Diagram for a *University* Database



Example: a Record Shop

- Entity sets
  - *customer*
  - *product* (CD, vinyl)
  - *purchase*
- Relationship sets
  - *contains* (between *product* and *purchase*)
  - *buyer* (between *customer* and *purchase*)

- An entity is a specific object; *E.g.*, a newest CD of BTS
- A relationship is specific pair of related objects

Example: a Record Shop

- Attributes
  - *customer*
    - *name, address, phone number, email, etc.*
  - *product*
    - *artist, title, price, description*
  - purchase
    - *date, payment_method*

  - Artificial primary keys for all entity sets

Example: a Record Shop

- Types of Relationships
  - 1-to-1
  - 1-to-many
    - *buyer* relation (between *customer* and *purchase*)
  - Many-to-many
    - *contains* relation (between *purchase* and *product*)

E-R Diagram

Example: Flight Database

- Entity sets
  - *airport*
    - *code, name, city*
  - *flight*
    - *flight_num, STD, STA, date_offset*
  - *departure*
    - *dept_date, capacity*
  - *customer*
    - *id, name, address, milege_num*
- Relationship sets
  - *to, from* (*flight, airport*)
  - *flight_dept (flight, departure)*
  - *reserved_on* (*customer, departure*)

E-R Diagram